Linguistic MDL Explanations with LLMs via Occam-Style Prompt Discovery

Yassine Meliani [1] **George Kontorousis** [1] **Eric Elmoznino** [2]

Abstract

Prompted LLMs can be viewed as natural-language models of data, where a prompt specifies the behavior of the system. However, explanation discovery with LLMs is typically heuristic, relying on manual prompt engineering or post-hoc generation, and lacks a principled trade-off between simplicity and predictive power. We study this problem through the lens of Minimum Description Length (MDL), where an explanation z is scored by a frozen language model using both its complexity $-\log p_*(z)$ and its ability to predict the data $-\log p_*(x \mid z)$. We train a policy model $p_\theta(z \mid x)$ with reinforcement learning to generate explanations that minimize this MDL objective. In a controlled synthetic setting based on number sequences, we find that standard conditional likelihood often favors fluent but incorrect explanations, while MDL-based fine-tuning shifts model preferences toward more concise and predictive rules. This results in improved recovery of underlying patterns, such as compact symbolic expressions, compared to baseline prompting. Despite limitations in likelihood estimation and exploration, our results provide evidence that MDL is a useful objective for guiding language models toward better explanations, suggesting a promising direction for combining generative modeling with structure discovery.

1. Introduction

Large language models can often be adapted to new tasks by changing only the prompt. This flexibility has motivated the view that prompts can function as natural-language parameters that shape model behavior (Petrov et al., 2024). Under this perspective, a prompt or free-form textual explanation can be seen not just as an instruction, but as a candidate hypothesis about the structure underlying observed data.

In practice, however, explanation discovery with LLMs remains largely heuristic. Prompt engineering is manual and sensitive to wording, while post-hoc explanations can be fluent without being genuinely predictive. What is often missing is a principled criterion for preferring explanations that are both simple and useful for prediction (Singh et al., 2024; Bordt et al., 2024; Zhao et al., 2023).

We study this problem through the lens of Minimum Description Length (MDL). In our setting, an explanation z is a natural-language hypothesis for a dataset x. We score an explanation using two terms: its complexity under a frozen language model prior, $-\log p_*(z)$, and the complexity of the data given that explanation, $-\log p_*(x \mid z)$. This yields an Occam-style objective that favors explanations that are concise while still making the data predictable.

Based on this objective, we train a policy model $p_\theta(z \mid x)$ to propose candidate explanations for observed data, while a frozen evaluator model p_* scores them. The goal is not simply to generate likely explanations under the policy, but to shift the policy toward explanations that achieve lower MDL under the fixed evaluator.

We evaluate this idea in a controlled synthetic setting based on number sequences with known underlying rules. This allows us to test whether MDL-based training helps recover compact, predictive explanations more reliably than standard conditional generation. We compare against prompting-based baselines and analyze both explanation quality under the frozen evaluator and the conditional preferences of the learned policy.

2. Related Works

Discovering explanations for observed data. The emergence of large language models (LLMs) has led to increasing interest in using them to generate linguistic explanations for patterns observed in data. LLMs are trained on massive amounts of data which grants them knowledge of multiple fields. Recent work has explored leveraging the world knowledge that LLMs have to analyze and explain many different kinds of data (Singh et al., 2024; Bordt et al., 2024; Zhao et al., 2023). These findings motivate further research

[1]Department of Computer Science, McGill University, Montreal, Canada [2]Mila Quebec. Correspondence to: Yassine Meliani <yassine.meliani@mail.mcgill.ca>, George Kontorousis <george.kontorousis@mail.mcgill.ca>, Eric Elmoznino <eric.elmoznino@mila.quebec>.

Preprint. April 22, 2026.

on optimizing LLMs as tools for discovering interpretable hypotheses and explanations for a multitude of scientific areas. A method to model learning and evaluation of hypotheses for observations is as a Bayesian inference over them, expressed in natural language. Ellis (2023) treat natural language as a hypothesis space, and generate candidate explanations for observed data. The candidates are evaluated using a Bayesian prior over the possible concepts and the likelihood of the observed examples given a candidate. The posterior distribution then identifies explanations that are both an explanation of the data and are plausible under the prior based on LLMs. Agarwal et al. (2026) study open-ended scientific discovery employing LLM agents that given observations are utilized to propose hypotheses. These hypotheses are evaluated with Bayesian surprise to guide research into possible directions that would have meaningful results.

Use of prompts as latent explanations of data. Linguistic explanations of data can be valuable for improving predictive performance. Singh et al. (2023) propose iPrompt, which searches for prompts that explain input–output relationships by iteratively generating candidate explanations and selecting those that best predict the observed data. In that framework, the candidate prompt serves as an explanation of the relationship between inputs and outputs. Our setting is related, but differs in that we focus on explaining the structure of observed number sequences rather than supervised input–output mappings. Moreover, our final implementation studies a single fixed synthetic dataset of sequence examples generated from known rules. Within this setting, we train a policy to generate candidate explanations for observed sequences and optimize them with an MDL objective that balances simplicity and predictive accuracy.

Prompt optimization for language models. The modeling of prompt as a tunable hyperparameter in LLMs is a deeply explored area. AutoPrompt (Shin et al., 2020) performs gradient-guided search over discrete trigger tokens inserted into prompt templates in order to maximize task likelihood. Subsequent work explores optimizing continuous prompts instead of discrete tokens; for example, prefix-tuning (Li & Liang, 2021) learns a sequence of continuous vectors prepended to the input that act as fine-tuning parameters that affect outputs while keeping the language model parameters fixed. More closely related to our work, approaches such as RLPrompt (Deng et al., 2022) formulate prompt discovery as a reinforcement learning problem and train a policy network to generate prompts that maximize downstream task rewards. These approaches demonstrate that prompts can be treated as optimization parameters that adapt a pretrained language model to a particular task. However, the objective in these methods is typically to improve task performance such as classification accuracy or generation quality rather than to discover interpretable explanations of the data. In our work, the natural language explanation can be indirectly seen as a prompt that explain the observed data. We learn a policy to generate explanations that maximize a MDL objective based on the data observed rather than downstream task performance.

Bayesian explanation discovery and amortized inference. Hu et al. (2023) show that many reasoning tasks can be interpreted as sampling latent text sequences from an intractable posterior distribution and propose training a policy model to directly generate samples from this posterior. By conditioning the policy on the input, the learned model amortizes the inference procedure and can efficiently produce latent reasoning sequences - explanations of the data - at test time. This shows that approximation of the posterior has better results than in-context learning or zero-shot generation and avoids the computational demand that methods such as beam search and chain-of-thought sampling have on inference time. Our work adopts a similar amortized perspective but applies it to the discovery of natural-language explanations for datasets. We train a policy to generate candidate explanations conditioned on the data and evaluate them using a description-length objective derived from the likelihood of a pretrained language model. In this formulation, the prior term over explanations favors shorter or simpler explanations, implementing an Occam-style preference for hypotheses that compress the data while remaining predictive.

3. Modeling

Our goal is to learn explanations that are not only likely under a language model, but also simple and predictive of the observed data. We compare standard conditional generation against an MDL-based training approach that explicitly favors this trade-off.

3.1. Baseline

Our baseline is zero-shot explanation generation with a pretrained language model used without additional task-specific training. Given observed data x, it generates an explanation z according to

$$p_{\text{base}}(z \mid x). \tag{1}$$

In practice, we use zero-shot greedy decoding under the conditional prompt described in Appendix B.

To score candidate explanations under this model, we use the conditional loss

$$\mathcal{L}^{\text{base}}_{\text{cond}}(z, x) = -\log p_{\text{base}}(z \mid x). \tag{2}$$

3.2. Proposed Model

MDL scoring. We evaluate explanations using a frozen evaluator model p^*. For a candidate explanation z and observed data x, we define the MDL loss

$$\mathcal{L}_{\mathrm{MDL}}(z, x) = -\lambda \log p^*(z) - \log p^*(x \mid z), \tag{3}$$

where the first term favors simpler explanations and the second favors explanations that make the data predictable. The choice of λ determines how strongly the objective pushes the model toward simpler and more plausible explanations, as opposed to explanations that primarily maximize predictive fit to the observed data.

Policy model. We introduce a trainable policy $\pi_\theta(z \mid x)$, initialized from the same pretrained model as the baseline and fine-tuned using LoRA. The evaluator p^* is kept fixed throughout training.

Optimization. For each input x, the policy samples candidate explanations z, which are scored using the MDL loss above. We define the reward as the negative MDL loss,

$$R(z, x) = -\mathcal{L}_{\mathrm{MDL}}(z, x), \tag{4}$$

and optimize the policy with REINFORCE:

$$\mathcal{L}_{\mathrm{policy}} = -(R(z, x) - b) \log \pi_\theta(z \mid x), \tag{5}$$

where b is a variance-reduction baseline taken to be the mean reward of the sampled explanations for the current input:

$$b = \frac{1}{K} \sum_{k=1}^{K} R(z_k, x).$$

Reported scores. We report three quantities throughout the paper: (1) the conditional loss under the base model, $-\log p_{\text{base}}(z \mid x)$; (2) the conditional loss under the fine-tuned policy, $-\log \pi_\theta(z \mid x)$; and (3) the MDL loss under the frozen evaluator, $\mathcal{L}_{\mathrm{MDL}}(z, x)$. The first two measure how strongly each model prefers a candidate explanation, while the third evaluates explanation quality under a fixed simplicity–predictiveness trade-off.

Training objective. The goal of training is to increase the probability of explanations that achieve low MDL loss under the frozen evaluator. In this sense, fine-tuning encourages the policy to assign higher probability to explanations that are both concise and predictive, without changing the evaluator itself.

4. Dataset and Evaluation

4.1. Data

Our original proposal considered natural-language datasets such as ScienceWorld (Wang et al., 2022) and related task-trace settings. In practice, however, we found it difficult to identify a dataset that simultaneously satisfied three requirements: (i) the data should admit a compact underlying rule, (ii) that rule should be expressible as a short explanation, and (iii) explanation quality should be easy to verify. We therefore focus on a controlled synthetic dataset of number sequences generated by hand-designed Python functions.

The dataset is constructed directly from a fixed library of 20 generator functions. For each generator, we take exactly one example: the first 10 terms produced by that rule. As a result, the dataset contains 20 sequence–rule pairs in total, with one sequence per rule. Each example consists of an observed sequence x together with a target explanation z describing the generating rule.

The generator library includes simple closed-form rules, recurrence-based patterns, digit-manipulation rules, and more irregular handcrafted constructions. Because the dataset is synthetic, the main object of interest is not a class label but the generating rule itself. This setting allows us to test whether the model recovers the intended rule rather than merely producing a plausible explanation. Full examples are provided in Appendix A.

4.2. Evaluation Metrics

We evaluate explanations using two quantities: conditional loss and MDL loss.

Conditional loss measures how strongly a model prefers a candidate explanation given an observed sequence. We report this quantity under both the base model and the fine-tuned policy.

MDL loss is the fixed evaluator objective introduced in Section 3. Our goal is not only to reduce conditional loss under the fine-tuned policy, but to shift the policy toward explanations that also achieve lower MDL loss. In this sense, we want the model's conditional preferences to move closer to the explanations favored by the MDL objective. This encourages explanations that both fit the data well and remain plausible under the model prior.

5. Experiments

All experiments were implemented in `PyTorch` using the `transformers` and `peft` libraries. We used `Qwen/Qwen2.5-7B-Instruct` as the base language model, with a LoRA-adapted copy as the trainable policy and a frozen copy as the evaluator. Prompts were formatted through the model's chat template with explicit system and user messages. Training and evaluation were performed in half precision (`bfloat16` when available) to reduce memory usage.

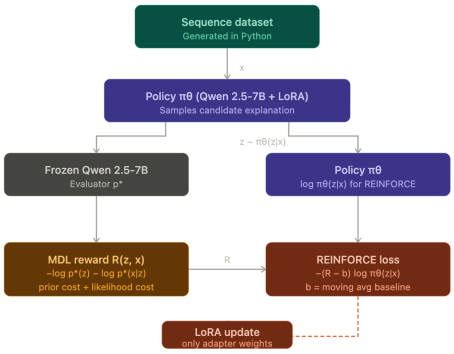

Figure 1. Overview of the training pipeline. The policy π_θ samples candidate explanations given a sequence, which are scored by a frozen evaluator using the MDL objective. The resulting reward is used to update the policy via REINFORCE, while only LoRA adapter weights are trained.

Training setup. The policy was trained with REINFORCE using the MDL reward described in Section 3. We used the AdamW optimizer with learning rate 5×10^{-5} for the multi-sequence setting, with larger learning rates used only in short diagnostic runs. Unless otherwise noted, we used 50 training episodes, 4 sampled explanations per example, sampling temperature 1.0, maximum generation length 64 tokens, gradient clipping with norm 1.0, and MDL weight $\lambda = 0.3$.

The overall training pipeline is illustrated in Figure 1.

Exploration. A key practical issue was exploration: correct explanations often had very low probability under the initial policy. To address this, we used an oracle-injection in which the ground-truth explanation was inserted with small probability during training. In our synthetic multi-sequence experiments, we used an injection probability of 0.25. This was not intended as the final method, but as a diagnostic to test whether the policy could reinforce correct explanations once they were observed.

Evaluation protocol. At evaluation time, explanations were generated with greedy decoding unless otherwise stated. We report three scores: conditional loss under the base model, conditional loss under the fine-tuned model, and MDL loss under the frozen evaluator. For consistency, all MDL numbers were computed with the same frozen evaluator and the same value of λ. In all reported experiments, we used $\lambda = 0.3$. Empirically, larger values placed too much weight on explanation simplicity and tended to favor overly simple rules, while smaller values reduced the influence of the prior and made overfitting more likely.

Runs and reporting. Because of the cost of running RL with a 7B parameter model, most experiments were conducted as focused proof-of-concept runs rather than large multi-seed sweeps. We therefore emphasize qualitative analysis, representative examples, and reward curves in addition to scalar metrics.

Implementation details. We observed that memory usage was a significant constraint, especially when simultaneously loading the base model, fine-tuned policy, and frozen evaluator. In practice, this required careful model reuse, mixed precision, and small sample counts per episode. We also found that greedy decoding does not necessarily recover the globally lowest conditional-loss explanation, so we distinguish between model outputs obtained by decoding and candidate rules scored post hoc.

Code. Code for data generation, training, and evaluation is available in our repo[1]

6. Results and Discussion

6.1. Results

We evaluate whether MDL-based fine-tuning changes the explanations preferred by the model. Our main comparison is between (i) the base model prompted directly for an explanation, and (ii) a LoRA fine-tuned policy trained with the frozen-evaluator MDL reward. Table 1 summarizes a representative example.

Explanation z	**Base cond.**	**FT cond.**	**MDL**
Base model greedy output	2.36	–	47.87
Fine-tuned greedy output	–	0.13	38.38
$2^n - n$ (true rule)	39.35	0.09	29.38
Max-likelihood rule	2.36	34.56	47.87
Dummy (copy data)	63.88	62.52	95.77

Table 1. Representative explanation scores for the sequence $(1, 1, 2, 5, 12, 27, 58, 121, 248, 503)$. Lower is better. In the first two rows, each model is evaluated on its own greedy-decoded output: the base conditional score is $-\log p_{\text{base}}(z_{\text{base}} \mid x)$ and the fine-tuned conditional score is $-\log \pi_\theta(z_{\text{ft}} \mid x)$. The remaining rows score fixed candidate explanations. MDL uses the frozen evaluator with $\lambda = 0.3$.

The baseline strongly prefers the recurrence-style explanation with the lowest conditional loss, even though it does not match the true generating rule. In contrast, the fine-tuned policy assigns much lower conditional loss to the true rule $2^n - n$, showing that training shifts model preference toward explanations favored by the MDL objective.

Figure 2 shows the training reward over REINFORCE updates. Although training is noisy, reward increases overall and stabilizes, indicating that the policy learns to propose

[1] https://github.com/gkontorousis/COMP-767

better explanations under the frozen evaluator.

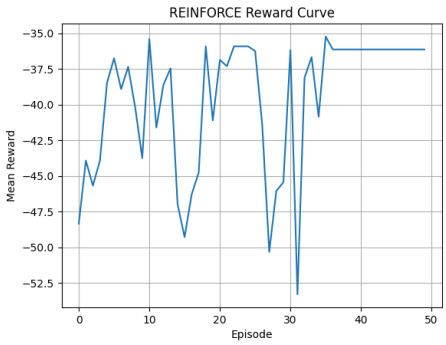

Figure 2. Mean reward per episode during REINFORCE training.

6.2. Discussion

Overall, the fine-tuned model does move in the intended direction: its conditional preferences become more aligned with the frozen MDL objective than those of the base model. In this sense, the proposed model improves over the baseline and partially confirms our hypothesis that conditional likelihood alone is not a sufficient criterion for explanation quality.

At the same time, the learned behavior was not exactly what we initially expected. We originally viewed MDL training as jointly improving both explanation simplicity and predictive quality. In practice, the strongest effect came from the data-fit term $p^*(x \mid z)$. The prior term $p^*(z)$ changed comparatively little: even before training, the model already tended not to assign high probability to highly unnatural or obviously implausible explanations. What changed most after fine-tuning was the model's ability to extract the underlying pattern from the observed data.

This also helps to explain the noise of the reward curve in Figure 2. The policy operates on discrete explanations in natural-language, and small changes in the sampled output can lead to qualitatively different rules. In turn, these changes can produce large shifts in either the complexity term or the data-fit term of the MDL objective. As a result, training does not evolve smoothly: reward can vary substantially across updates even when the underlying policy changes only modestly. Still, the overall trend suggests that the policy moves toward better explanations and eventually settles into locally preferred regions of the hypothesis space.

This suggests that, in our setting, MDL training acts less as a mechanism for teaching the model to prefer concise language, and more as a mechanism for improving pattern recovery while using the prior as a regularizer. That regularization remains important. If we were to optimize only the data-fit term, the model would have a stronger incentive to overfit, for example by memorizing the observed sequence or preferring explanations that match the prefix but do not generalize. The prior therefore still plays a useful role, even if it is not the dominant source of improvement.

6.3. Qualitative Analysis

The clearest failure case of the baseline is that it often produces fluent recurrence-style explanations that sound plausible but are mathematically incorrect. For example, for the sequence

$$(1, 1, 2, 5, 12, 27, 58, 121, 248, 503)$$

the base model explains the pattern as "Each term is the sum of the previous term and twice the term before it." This rule receives a very low conditional loss under the base model, but a much worse MDL score than the true rule.

The fine-tuned model succeeds in cases like this by assigning a substantially lower conditional loss to the compact symbolic explanation $2^n - n$. This is the main success of the method: it shifts the model away from what is merely likely to be said and toward what better explains the data.

However, the fine-tuned model does not always decode the best global explanation. In some runs, greedy decoding produced near-miss explanations such as $2^n - n - 1$, which are close to the true rule but still suboptimal under the frozen evaluator. This reveals an important distinction between the scoring function and the decoding procedure: even when the correct rule has a lower conditional loss than competing explanations, greedy decoding may still fail to recover it.

We also found a more fundamental limitation when moving beyond synthetic number sequences. On more open-ended natural-language data, token-level likelihoods became much harder to interpret as explanation quality. In those cases, the MDL signal was much noisier, and the model's scores were less clearly tied to the correctness of the explanation. This was one reason we ultimately focused on synthetic sequences as a controlled setting.

Finally, our oracle-injection showed that once the correct explanation is observed during training, the policy can reinforce it and make it highly preferred under the fine-tuned conditional objective. This suggests that the main bottleneck is not the reward definition itself, but exploration: the correct explanation is often hard to discover from the initial model distribution.

6.4. Limitations and Challenges

While the MDL-based approach shows promising behavior in the controlled synthetic setting, several limitations remain.

Likelihood estimation. Accurately estimating $p(x \mid z)$ using a language model is challenging, especially for tasks

requiring precise reasoning such as arithmetic. Without chain-of-thought or structured intermediate reasoning, small and medium-scale models often assign noisy likelihoods to sequences, even when the underlying rule is correct. As a result, the MDL signal can be imperfect, particularly for explanations that require exact computation. Incorporating structured reasoning or chain-of-thought prompting could improve likelihood estimation, but was not explored here due to computational constraints.

Scaling to natural language. When moving beyond synthetic number sequences to more open-ended natural-language data (e.g., ScienceWorld-style traces), MDL scores become less informative. Token-level probabilities are diffuse, and differences between candidate explanations are often small and poorly correlated with explanation correctness. This motivated our focus on synthetic sequences, where correctness is well-defined and likelihood differences are more meaningful.

Exploration and discovery. A central challenge is that correct explanations often lie in low-probability regions of the initial model distribution. While policy optimization can improve locally likely candidates, it rarely discovers entirely new explanations without guidance, which limits the ability of REINFORCE to explore the hypothesis space effectively in practice. To better understand this issue, we introduced an oracle-injection in which the ground-truth explanation was occasionally provided during training with probability 0.25. Once observed, the policy reliably reinforced and preferred the correct explanation under the MDL objective. This suggests that the main bottleneck is not the reward formulation itself, but the difficulty of discovering good explanations through exploration alone.

Overall, these limitations highlight that while MDL provides a useful objective for evaluating explanations, successful explanation discovery depends critically on both reliable likelihood estimation and effective exploration strategies.

7. Future Directions

Our results highlight several promising directions for future work.

Improved likelihood estimation. A key limitation of our approach is the difficulty of accurately estimating $p(x \mid z)$ using standard next-token likelihoods. One natural extension is to incorporate chain-of-thought or structured reasoning when evaluating candidate explanations. By allowing the model to reason explicitly about intermediate steps, this could yield more reliable likelihood estimates for explanations that require precise computation.

Better exploration strategies. Our experiments show that the main bottleneck is not the MDL reward itself, but the difficulty of discovering correct explanations through sampling. In this work, we used oracle injection as a diagnostic tool to verify that correct explanations can be reinforced once observed. A more principled approach would be to design exploration strategies that reduce reliance on such guidance, for example through structured search, proposal distributions, or alternative reinforcement learning paradigms that better cover low-probability regions of the hypothesis space.

Scaling to richer datasets. While synthetic number sequences provide a controlled setting, extending this framework to natural-language data remains an important challenge. Future work should investigate settings where explanations remain verifiable while being expressed in natural language, as well as improved scoring methods that make MDL-style objectives meaningful in these domains.

Overall, advancing this line of work will require jointly improving evaluation (likelihood estimation) and search (exploration), rather than focusing on either component in isolation.

8. Conclusion

We studied explanation discovery with language models through the lens of Minimum Description Length. Our central finding is that standard conditional likelihood and explanation quality are not the same: a pretrained LLM often prefers explanations that are fluent and locally plausible, even when they do not match the true underlying rule. By contrast, an MDL-based objective provides a more useful target because it explicitly balances explanation simplicity and predictive fit.

Using a frozen evaluator and a LoRA fine-tuned policy, we showed that reinforcement learning can shift model preferences toward explanations with lower MDL loss. In our synthetic number-sequence setting, this led to substantially better recovery of compact symbolic rules such as $2^n - n$. At the same time, our experiments also revealed two important limitations. First, much of the gain came from improving the data-fit term rather than substantially changing the prior over explanations. Second, the main practical bottleneck was exploration: the correct explanation often lies in a low-probability region of the initial policy and is difficult to discover without additional guidance.

More broadly, our results suggest that MDL is a promising framework for training language models to produce better explanations, but that successful explanation discovery depends on both the quality of the evaluator and the search procedure used during training and decoding. Future work should study stronger evaluators, better exploration

strategies, and more realistic datasets in which explanations remain both verifiable and linguistically meaningful.

Despite the limited scale of our experiments and the constraints imposed by computational resources, our results provide evidence that language models can, when trained with an appropriate MDL objective, move toward extracting meaningful explanations from data rather than merely producing plausible text. This suggests a broader direction in which LLMs are not only generative models of language, but also tools for discovering compact structure in data. Future work may explore richer MDL formulations, improved likelihood estimation (e.g., via structured reasoning), and even connections to compression-based perspectives, where explanations act as concise programs that effectively "compress" observed data.

Author Contributions. George Kontorousis and Yassine Meliani contributed equally to the conceptualization, implementation, experimentation, analysis, and writing of this project. Eric Elmoznino provided supervision and guidance on the research direction, methodology, and presentation of results.

References

Agarwal, D., Majumder, B. P., Adamson, R., Chakravorty, M., Gavireddy, S. R., Parashar, A., Surana, H., Mishra, B. D., McCallum, A., Sabharwal, A., and Clark, P. Autodiscovery: Open-ended scientific discovery via bayesian surprise, 2026. URL https://arxiv.org/abs/2507.00310.

Bordt, S., Lengerich, B. J., Nori, H., and Caruana, R. Data science with llms and interpretable models. *ArXiv*, abs/2402.14474, 2024. doi: 10.48550/arxiv.2402.14474.

Deng, M., Wang, J., Hsieh, C.-P., Wang, Y., Guo, H., Shu, T., Song, M., Xing, E. P., and Hu, Z. Rlprompt: Optimizing discrete text prompts with reinforcement learning, 2022. URL https://arxiv.org/abs/2205.12548.

Ellis, K. Human-like few-shot learning via bayesian reasoning over natural language. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 13149–13178. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/2aa9b18b9ab37b0ab1fdaae46fb781d4-Paper-Conference.pdf.

Hu, E. J., Jain, M., Elmoznino, E., Kaddar, Y., Lajoie, G., Bengio, Y., and Malkin, N. Amortizing intractable inference in large language models, 2023. URL https://arxiv.org/abs/2310.04363.

Li, X. L. and Liang, P. Prefix-tuning: Optimizing continuous prompts for generation. In Zong, C., Xia, F., Li, W., and Navigli, R. (eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 4582–4597, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.353. URL https://aclanthology.org/2021.acl-long.353/.

Petrov, A., Torr, P. H. S., and Bibi, A. Prompting a pretrained transformer can be a universal approximator, 2024. URL https://arxiv.org/abs/2402.14753.

Shin, T., Razeghi, Y., Logan IV, R. L., Wallace, E., and Singh, S. AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts. In Webber, B., Cohn, T., He, Y., and Liu, Y. (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 4222–4235, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.346. URL https://aclanthology.org/2020.emnlp-main.346/.

Singh, C., Morris, J. X., Aneja, J., Rush, A., and Gao, J. Explaining data patterns in natural language with language models. In Belinkov, Y., Hao, S., Jumelet, J., Kim, N., McCarthy, A., and Mohebbi, H. (eds.), *Proceedings of the 6th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pp. 31–55, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.blackboxnlp-1.3. URL https://aclanthology.org/2023.blackboxnlp-1.3/.

Singh, C., Inala, J., Galley, M., Caruana, R., and Gao, J. Rethinking interpretability in the era of large language models. *ArXiv*, abs/2402.01761, 2024. doi: 10.48550/arxiv.2402.01761.

Wang, R., Jansen, P., Côté, M.-A., and Ammanabrolu, P. ScienceWorld: Is your agent smarter than a 5th grader? In Goldberg, Y., Kozareva, Z., and Zhang, Y. (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 11279–11298, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.775. URL https://aclanthology.org/2022.emnlp-main.775/.

Zhao, H., Chen, H., Yang, F., Liu, N., Deng, H., Cai, H., Wang, S., Yin, D., and Du, M. Explainability for large

language models: A survey. *ACM Transactions on Intelligent Systems and Technology*, 15:1 – 38, 2023. doi: 10.1145/3639372.

A. Number Sequences

Below we list representative number sequences from our synthetic dataset together with concise linguistic descriptions of their generating rules.

- **Sequence:** $1, 4, 7, 10, 13, 16, 19, 22, 25, \ldots$
  Rule: $y = 3n + 1$.
- **Sequence:** $2, 4, 8, 16, 32, 64, 128, 256, 512, \ldots$
  Rule: Geometric progression with ratio 2.
- **Sequence:** $1, 1, 2, 5, 12, 27, 58, 121, 248, \ldots$
  Rule: $2^n - n$, with indexing starting at $n = 0$.
- **Sequence:** $1, 4, 9, 16, 25, 36, 49, 64, 81, \ldots$
  Rule: n^2.
- **Sequence:** $2, 4, 8, 16, 32, 64, 128, 256, 512, \ldots$
  Rule: 2^{n+1}.
- **Sequence:** $1, 3, 6, 10, 15, 21, 28, 36, 45, \ldots$
  Rule: $\frac{n(n+1)}{2}$.
- **Sequence:** $2, 4, 7, 11, 16, 22, 29, 37, 46, \ldots$
  Rule: $1 + \frac{n(n+1)}{2}$.
- **Sequence:** $0, 2, 6, 12, 20, 30, 42, 56, 72, \ldots$
  Rule: $n(n-1)$.
- **Sequence:** $0, 1, 1, 2, 3, 5, 8, 13, 21, \ldots$
  Rule: Fibonacci numbers.
- **Sequence:** $2, 3, 5, 7, 11, 13, 17, 19, 23, \ldots$
  Rule: Prime numbers.
- **Sequence:** $1, 22, 333, 4444, 55555, \ldots$
  Rule: Repeat digit n exactly n times.
- **Sequence:** $0, 2, 6, 9, 11, 14, 16, 19, 21, \ldots$
  Rule: Start at 0, then add increments cycling through $2, 3, 4, 3, 2$.
- **Sequence:** $2, 4, 7, 8, 10, 13, 14, 16, 19, \ldots$
  Rule: Start from $n = 1$. Output $2n$, except every third term, where the value is $2n + 1$.
- **Sequence:** $11, 11, 12, 13, 15, 18, 23, 31, 44, 65, \ldots$
  Rule: Start at 11, then repeatedly add successive Fibonacci numbers.
- **Sequence:** $4, 36, 900, 44100, 5336100, \ldots$
  Rule: The cumulative product of squared primes:
  $$\prod_{k=1}^{n} p_k^2.$$
- **Sequence:** $2, 12, 180, 6300, 485100, \ldots$
  Rule: Multiply by each prime twice before moving to the next prime.
- **Sequence:** $4, 4, 36, 8, 100, 12, 196, 16, 324, \ldots$
  Rule: Odd-indexed terms are $(2n)^2$, while even-indexed terms are $2n$.

- **Sequence:** $1, 24, 24, 31, 120, 120, 97, 288, 288, \ldots$
  Rule: A piecewise sequence depending on $n \bmod 3$:

$$\begin{cases} 2n^2 - 1, & n \equiv 1 \pmod 3, \\ (2n+1)^2 - 1, & n \equiv 2 \pmod 3, \\ (2n-1)^2 - 1, & n \equiv 0 \pmod 3. \end{cases}$$

- **Sequence:** $1, 11, 21, 1211, 111221, 312211, \ldots$
  Rule: Look-and-say sequence: each term describes the runs of digits in the previous term.
- **Sequence:** $3, 15, 27, 1219, 111231, \ldots$
  Rule: $2n +$ look-and-say(n).

B. Prompt Templates

In all experiments, prompts were formatted as two-message chats (a system message followed by a user message) using the tokenizer's chat template. The shared system instruction used for conditional explanation generation was:

> **System:** You are a careful pattern-discovery assistant. Given an observed number sequence, propose a concise natural-language rule that likely generated it. Return only the rule, with no extra commentary.

We then instantiated three prompt templates corresponding to the three scoring terms used in the paper.

Prior term: $-\log p(z)$ This prompt is used to evaluate the simplicity or plausibility of a candidate explanation z:

> **System:** `You are a careful pattern-discovery assistant. Write a concise rule for a number sequence. Return only the rule.`
>
> **User:** `Write a rule to generate a number sequence.`
>
> **Assistant:** {`z`}

Likelihood term: $-\log p(x \mid z)$ This prompt is used to evaluate how well an explanation predicts the observed sequence:

> **System:** `You are a careful sequence predictor. Given a rule, continue the number sequence implied by that rule.`
>
> **User:** `Rule:`
>
> {`z`}
>
> `Output the first 10 terms of the sequence as comma-separated integers only.`
>
> **Assistant:** {`x`}

Conditional baseline: $-\log p(z \mid x)$ This prompt is used for standard explanation generation from the observed sequence:

> **System:** `You are a careful pattern-discovery assistant. Given an observed number sequence, propose a concise natural-language rule that likely generated it. Return only the rule, with no extra commentary.`
>
> **User:** `Observed sequence:`
>
> `1, 1, 2, 5, 12, 27, 58, 121, 248, 503`
>
> `What is the most likely concise rule that generated this sequence?`
>
> **Assistant:** {`z`}

Here, x denotes the observed sequence and z denotes the candidate explanation.